

Maximizing XOR

Given two integers, l and r , find the maximal value of $a \text{ xor } b$, where a and b satisfy the following condition:

$$l \leq a \leq b \leq r$$

For example, if $l = 11$ and $r = 12$, then

$$\begin{aligned} 11 \wedge 11 &= 0 \\ 11 \wedge 12 &= 7 \\ 12 \wedge 12 &= 0 \end{aligned}$$

Our maximum value is 7.

Numbers after powers of two follow a pattern.

Example after 8:

$$\begin{aligned} 8 &= 1000 \\ 9 &= 1001 \\ 10 &= 1010 \\ 11 &= 1011 \\ 12 &= 1100 \\ 13 &= 1101 \\ 14 &= 1110 \\ 15 &= 1111 \end{aligned}$$

So once you know 8, you instantly know 9–15.

The $O(1)$ Algorithm

1. Find the XOR of L and R ($L \wedge R$). This instantly zeros out the shared prefix and leaves a 1 at the exact divergence point.
2. Find the position of that highest 1 bit.
3. The maximum possible XOR is simply a number where that bit, and all bits below it, are 1.

Code

```
#include <bits/stdc++.h>
using namespace std;

int solve(int l, int r) {
    int XOR_result = l ^ r;

    // If l and r are the same, XOR is 0, and the max is 0.
    if (XOR_result == 0) return 0;

    // __builtin_clz returns the number of leading zeros in a 32-bit integer.
    // 32 - __builtin_clz gives us the number of bits required to represent the number.
    int num_bits = 32 - __builtin_clz(XOR_result);

    // We want a number with 'num_bits' ones.
    // Shifting 1 left by num_bits gives us a power of 2.
    // Subtracting 1 gives us a solid block of 1s.
    return (1 << num_bits) - 1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int l, r;
    cin >> l >> r;

    cout << solve(l, r) << "\n";
    return 0;
}
```